

Constraining GLM trends from MMEs using observations: 3 regression parameter example

Introduction

Let's repeat the work from `example.rmd` but instead using a length 3 vector for b^j , $j = 1, \dots, m$ and a .

Instead of just considering a Poisson trend with an intercept and slope, lets consider a Gamma distribution with an intercept, slope and shape.

Then the observations are distributed as a Gamma glm:

$$z_t | \nu(t) \sim \text{Gamma}(\alpha_z, \beta_z(t)), \quad t = t_1, \dots, t_{n_h}, \quad (1)$$

$$\log\left(\frac{\alpha_z}{\beta_z(t)}\right) = a_1 + a_2 t, \quad (2)$$

$$\log(\alpha_z) = a_3. \quad (3)$$

And similarly for the climate simulations:

$$x_t^j | \nu(t) \sim \text{Gamma}(\alpha_{x^j}, \beta_{x^j}(t)), \quad t = t_1, \dots, t_{n_h}, \quad (4)$$

$$\log\left(\frac{\alpha_{x^j}}{\beta_{x^j}(t)}\right) = b_1^j + b_2^j t, \quad (5)$$

$$\log(\alpha_{x^j}) = b_3^j. \quad (6)$$

The similarity parameter in N-dimensions

As the vector δ gets longer, the framework becomes progressively more difficult to fit in stan. In our testing for $\text{len}(\delta) > 2$, we get divergent transitions and maximum treedepth issues. It is also harder to visualise and interpret higher dimensional vectors.

As an alternative to having δ as a vector we could assume that all components of δ are the same and replace it with a single scalar determining the similarity between the ensemble and the real world. Then:

$$\Gamma = \frac{1 - \delta}{\delta} \text{diag}(\Sigma). \quad (7)$$

This corresponds to the belief that all of the components of a have the same ratio of variances with the ensemble and removes the ability to consider one component as exchangeable with the ensemble but another as independent, for example.

Simulated data

Again set up the time covariate, t in years, which we rescale to t_sc and generate some simulated parameters (intercept and slope and $\log(\text{shape})$) where we know the true Σ , μ and w . In this example we assume we have a longer observational period.

```

start_date <- 1850
end_hist <- 2014
end_date <- 2100
nh <- end_hist - start_date + 1
nf <- end_date - start_date + 1
t <- seq(start_date, end_date, length.out = nf)
t_sc <- (t - start_date)/(nf - 1)

m = 20 # number of simulations
set.seed(1234)

Sigma_true <- cbind(c(.02,-.01, 0),c(-.01,.2, 0), c(0, 0, 0.1))
mu_true <- cbind(2, 0, 0)
L_true <- t(chol(Sigma_true))

# The true simulation regression parameters
b_true <- matrix(rep(mu_true, m), nrow = length(mu_true)) + L_true %*% matrix(rnorm(m * length(mu_true)),
w_true <- c(0.5,-0.3, 0)

# The true real world regression parameters
a_true <- as.numeric(mu_true) + L_true %*% rnorm(length(mu_true))# + w_true

print("mu is:")

## [1] "mu is:"
print(mu_true)

##      [,1] [,2] [,3]
## [1,]    2    0    0
print("Sigma is:")

## [1] "Sigma is:"
print(Sigma_true)

##      [,1] [,2] [,3]
## [1,]  0.02 -0.01  0.0
## [2,] -0.01  0.20  0.0
## [3,]  0.00  0.00  0.1

Now lets simulate the data as a Gamma distributed GLM.

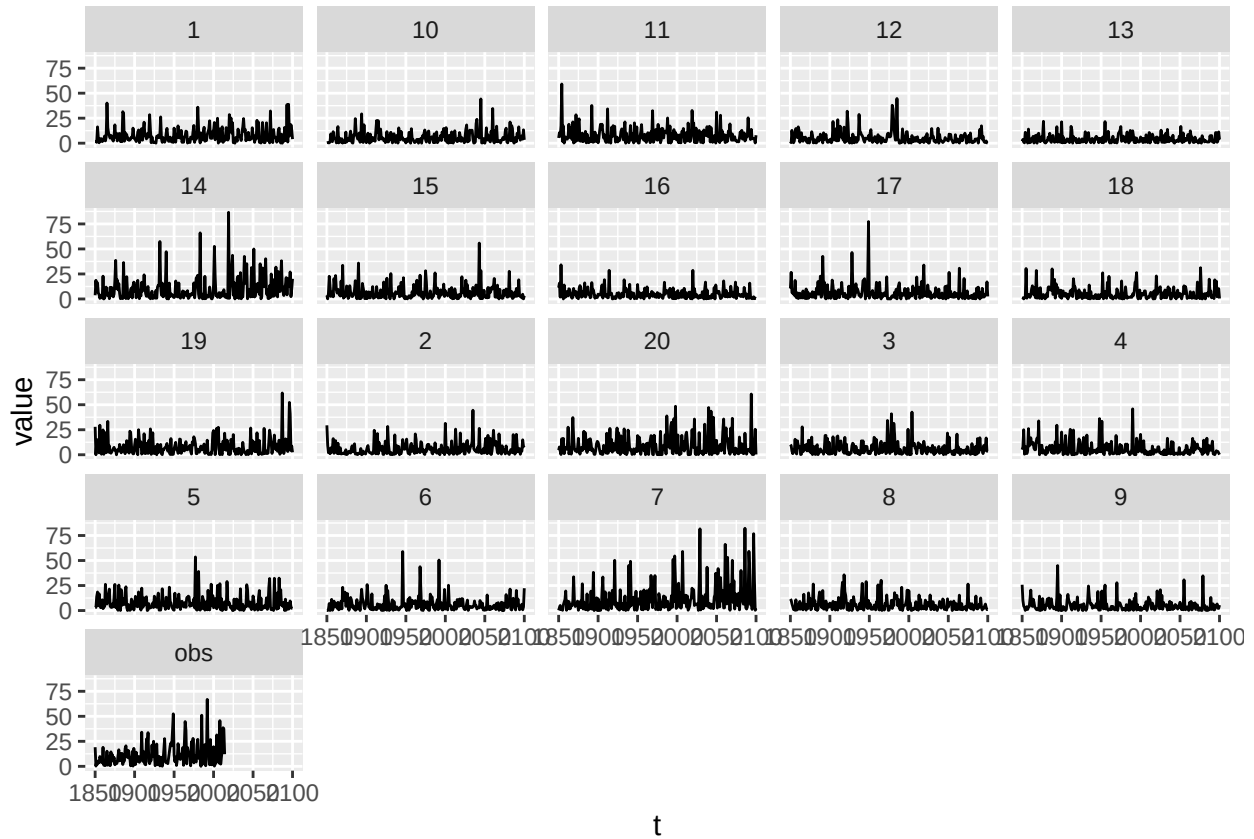
B <- cbind(rep(1, nf), t_sc)

X <- matrix(rgamma(n = m * nf, exp(b_true[3,]), exp(b_true[3,])/exp(as.numeric(B %*% b_true[1:2,]))), n
z <- rgamma(nh, exp(a_true[3]), exp(a_true[3])/exp(as.numeric(B[1:nh,] %*% a_true[1:2])))

data_df <- data.frame(X) %>%
  mutate(t_sc = t_sc, t = t) %>%
  pivot_longer(-c(t_sc, t),names_to="model", names_prefix="X") %>%
  bind_rows(
    data.frame(value = z, t_sc = t_sc[1:nh], t = t[1:nh], model = "obs")
  )

```

```
data_df %>%
  ggplot(aes(x = t, y = value)) +
  geom_line() +
  facet_wrap(~model)
```



Fitting the model

As in the previous example we do a maximum likelihood step. However as we need the shape parameter to be fitted using a log link we use the `glmmTMB` package instead of the base R `glm` function. For the Gamma distribution using the `glmmTMB` a log link for the shape (dispersion) parameter is the default.

```
fit_list <- list()
for(j in 1:m){
  fit_list[[j]] <- glmmTMB(
    value ~ t_sc,
    family=Gamma(link = "log"),
    filter(data_df, model == j)
  )
}
a_fit <- glmmTMB(value ~ t_sc, family=Gamma(link = "log"), filter(data_df, model == "obs"))

b_hat <- matrix(ncol = length(mu_true), nrow = m)
J <- array(0, dim = c(m, length(mu_true), length(mu_true)))
for(j in 1:m){
  bfit <- fit_list[[j]]
  b_hat[j,] <- c(unname(fixef(bfit)$cond), unname(fixef(bfit)$disp))
}
```

```

    J[j,,] <- t(chol(vcov(fit_list[[j]]), full = TRUE)))
  }
a_hat <- c(unnamed(fixef(a_fit)$cond),unnamed(fixef(a_fit)$disp))
J_a <- t(chol(vcov(a_fit, full = TRUE)))

```

Different values of delta

Consider a number of different δ values. Note the model cannot accept a value of $\delta = 0$.

```
deltas = c(0.001,seq(.25,1,.25))
```

```

prior_hyper_param = numeric()           # an empty vector will result in no priors
# prior_hyper_param = c(
#   mu_mu_1 = 0,
#   mu_mu_2 = 0,
#   mu_mu_3 = 0,
#   mu_sd_1 = 10,
#   mu_sd_2 = 5,
#   mu_sd_3 = 5,
#   eta = 2,
#   gam = 2.5
# )   # a vector of prior hyper parameters

```

```

data_list_mult_delta <- list(
  len_a = 3,
  m = m,
  b_hat = b_hat,
  a_hat = a_hat,
  J = J,
  J_a = J_a,
  len_hyper_param = length(prior_hyper_param),
  hyper_param = prior_hyper_param,
  n_delta = length(deltas),
  delta = deltas                                # Fixed values of delta
)

```

```

# This stan model fits the hierarchical model for a set of fixed deltas.
mod <- cmdstan_model("stan/glm_constr_mme_obs_fixed_1D_delta_vector.stan")

```

```

data_list_mult_delta[["delta"]] <- deltas
data_list_mult_delta[["n_delta"]] <- length(deltas)

```

```

fit_fixed_delta_mult_delta <- mod$sample(
  data = data_list_mult_delta,
  seed = 1234,
  iter_warmup = 1000,
  iter_sampling = 1000,
  chains = 4,
  parallel_chains = 4,
  refresh = 0,
  adapt_delta=.95,
  show_exceptions = FALSE
)

```

```
## Running MCMC with 4 parallel chains...
##
## Chain 2 finished in 18.4 seconds.
## Chain 1 finished in 18.7 seconds.
## Chain 4 finished in 18.5 seconds.
## Chain 3 finished in 19.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 18.7 seconds.
## Total execution time: 19.6 seconds.
```

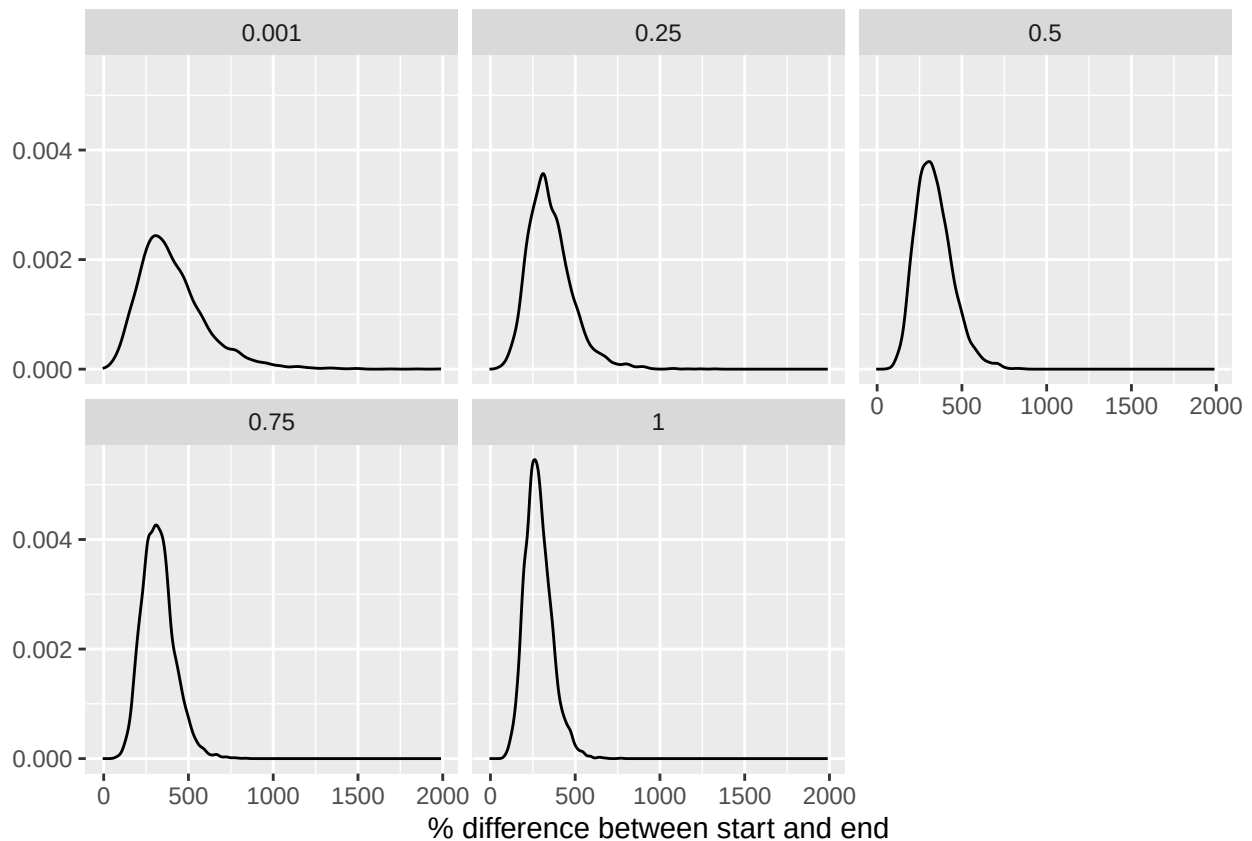
Lets use precentage change from the start of the time period to the end of the time period ($t_{sc} = 0$, $t = 1850$ to $t_{sc} = 1$, $t = 2100$) as an interesting example:

$$100\% \frac{E(z|t = 2100) - E(z|t = 1850)}{E(z|t = 1850)} = 100\% \frac{\exp(a_1 + a_2) - \exp(a_1)}{\exp(a_1)} \quad (8)$$

$$= 100\%(\exp(a_2) - 1) \quad (9)$$

Plotting for the different choices of δ

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Again this can be done with a prior on δ , e.g. $\delta \sim \text{Unif}(0, 1)$.

```
prior_hyper_param = numeric() # an empty vector will result in no priors
# prior_hyper_param = c(
#   mu_mu_1 = 0,
```

```

#      mu_mu_2 = 0,
#      mu_sd_1 = 10,
#      mu_sd_2 = 5,
#      eta = 2,
#      gam = 2.5
# )      # an length 6 vector of prior hyper parameters

data_list <- list(
  len_a = 3,
  m = m,
  b_hat = b_hat,
  a_hat = a_hat,
  J = J,
  J_a = J_a,
  len_hyper_param = length(prior_hyper_param),
  hyper_param = prior_hyper_param,
  delta_hyper_param = c(1,1)      # Prior parameters for delta ~ Beta() prior
)

mod <- cmdstan_model("stan/glm_constr_mme_obs_1D_delta.stan")

fit_prior_on_delta <- mod$sample(
  data = data_list,
  seed = 1234,
  iter_warmup = 1000,
  iter_sampling = 1000,
  chains = 4,
  parallel_chains = 4,
  refresh = 0,
  show_exceptions = FALSE
)

## Running MCMC with 4 parallel chains...
##
## Chain 3 finished in 7.6 seconds.
## Chain 4 finished in 7.8 seconds.
## Chain 2 finished in 8.1 seconds.
## Chain 1 finished in 8.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 7.9 seconds.
## Total execution time: 8.3 seconds.

## Warning: Dropping 'draws_df' class as required metadata was removed.

```

